

实践要求 3.1 与函数声明有关，函数声明需要在语法中添加产生式，并在符号表中记录每个函数当前的状态：是被实现了，还是只被声明未被实现。实践要求 3.2 涉及作用域，作用域的实现方法前文已经讨论过。实践要求 3.3 为实现结构等价，对于结构等价来说，只需要在判断两个类型是否相等时不是直接去比较类型名，而是针对结构体中的每个域逐个进行类型比较即可。

3.3 语义分析的实践内容

3.3.1 实践要求

在本次实践内容中，我们对 C 语言做如下假设，可以认为这些就是 C 语言的特性（注意，假设 3、4、5 可能因后面的不同选做要求而有所改变）：

- （1）假设 1：整型（int）变量不能与浮点型（float）变量相互赋值或者相互运算。
- （2）假设 2：仅有 int 型变量才能进行逻辑运算或者作为 if 和 while 语句的条件；仅有 int 型和 float 型变量才能参与算术运算。
- （3）假设 3：任何函数只进行一次定义，无法进行函数声明。
- （4）假设 4：所有变量（包括函数的形参）的作用域都是全局的，即程序中所有变量均不能重名。
- （5）假设 5：结构体间的类型等价机制采用名等价（Name Equivalence）的方式。
- （6）假设 6：函数无法进行嵌套定义。
- （7）假设 7：结构体中的域不与变量重名，并且不同结构体中的域互不重名。

以上七个假设也可视为要求，违反即会导致各种语义错误，不过我们只对后面讨论的 17 种错误类型进行考察。此外，可以安全地假设输入文件中不包含注释、八进制数、十六进制数、以及指数形式的浮点数，也不包含任何词法或语法错误（除了特别说明的针对选做要求的测试）。

程序需要对输入文件进行语义分析（输入文件中可能包含函数、结构体、一维和高维数组）并检查如下类型的错误：

- （1）错误类型 1：变量在使用时未经定义。
- （2）错误类型 2：函数在调用时未经定义。

- (3) 错误类型 3: 变量出现重复定义, 或变量与前面定义过的结构体名字重复。
- (4) 错误类型 4: 函数出现重复定义 (即同样的函数名出现了不止一次定义)。
- (5) 错误类型 5: 赋值号两边的表达式类型不匹配。
- (6) 错误类型 6: 赋值号左边出现一个只有右值的表达式。
- (7) 错误类型 7: 操作数类型不匹配或操作数类型与操作符不匹配 (例如整型变量与数组变量相加减, 或数组 (或结构体) 变量与数组 (或结构体) 变量相加减)。
- (8) 错误类型 8: `return` 语句的返回类型与函数定义的返回类型不匹配。
- (9) 错误类型 9: 函数调用时实参与形参的数目或类型不匹配。
- (10) 错误类型 10: 对非数组型变量使用 “[...]” (数组访问) 操作符。
- (11) 错误类型 11: 对普通变量使用 “(···)” 或 “()” (函数调用) 操作符。
- (12) 错误类型 12: 数组访问操作符 “[...]” 中出现非整数 (例如 `a[1.5]`)。
- (13) 错误类型 13: 对非结构体型变量使用 “.” 操作符。
- (14) 错误类型 14: 访问结构体中未定义过的域。
- (15) 错误类型 15: 结构体中域名重复定义 (指同一结构体中), 或在定义时对域进行初始化 (例如 `struct A {int a=0;}`)。
- (16) 错误类型 16: 结构体的名字与前面定义过的结构体或变量的名字重复。
- (17) 错误类型 17: 直接使用未定义过的结构体来定义变量。

其中, 要注意三点: 一是关于数组类型的等价机制, 同 C 语言一样, 只要数组的基类型和维数相同我们即认为类型是匹配的, 例如 `int a[10][2]` 和 `int b[5][3]` 即属于同一类型; 二是我们允许类型等价的结构体变量之间的直接赋值 (见后面的测试样例), 这时的语义是, 对应的域相应赋值 (数组域也如此, 按相对地址赋值直至所有数组元素赋值完毕或目标数组域已经填满); 三是对于结构体类型等价的判定, 每个匿名的结构体类型我们认为均具有一个独有的隐藏名字, 以此进行名等价判定。

除此之外, 程序可以选择完成以下部分或全部的要求:

(1) 要求 3.1: 修改前面的 C—语言假设 3, 使其变为“函数除了在定义之外还可以进行声明”。函数的定义仍然不可以重复出现, 但函数的声明在相互一致的情况下可以重复出现。任一函数无论声明与否, 其定义必须在源文件中出现。在新的假设 3 下, 程序还需要检查两类新的错误和增加新的产生式:

(a) 错误类型 18: 函数进行了声明, 但没有被定义。

(b) 错误类型 19: 函数的多次声明互相冲突(即函数名一致, 但返回类型、形参数量或者形参类型不一致), 或者声明与定义之间互相冲突。

(c) 由于 C—语言文法中并没有与函数声明相关的产生式, 因此需要先对该文法进行适当修改。对于函数声明来说, 我们并不要求支持像“`int foo(int, float)`”这样省略参数名的函数声明。在修改的时候要留意, 改动应该以不影响其它错误类型的检查为原则。

(2) 要求 3.2: 修改前面的 C—语言假设 4, 使其变为“变量的定义受可嵌套作用域的影响, 外层语句块中定义的变量可在内层语句块中重复定义(但此时在内层语句块中就无法访问到外层语句块的同名变量), 内层语句块中定义的变量到了外层语句块中就会消亡, 不同函数体内定义的局部变量可以相互重名”。在新的假设 4 下, 完成错误类型 1 至 17 的检查。

(3) 要求 3.3: 修改前面的 C—语言假设 5, 将结构体间的类型等价机制由名等价改为**结构等价**(Structural Equivalence)。例如, 虽然名称不同, 但两个结构体类型 `struct a {int x; float y;}` 和 `struct b {int y; float z;}` 仍然是等价的类型。注意, 在结构等价时不要将数组展开来判断, 例如 `struct A {int a; struct {float f; int i;} b[10];}` 和 `struct B {struct {int i; float f;} b[10]; int b;}` 是不等价的。在新的假设 5 下, 完成错误类型 1 至 17 的检查。

3.3.2 输入格式

程序的输入是一个包含 C—源代码的文本文件, 该源代码中可能会有语义错误。程序需要能够接收一个输入文件名作为参数。例如, 假设程序名为 `cc`、输入文件名为 `test1`、程序和输入文件都位于当前目录下, 那么在 Linux 命令行下运行 `./cc test1` 即可获得以 `test1` 作为输入文件的输出结果。

3.3.3 输出格式

实践内容二要求通过标准输出打印程序的运行结果。对于那些没有语义错误的输入文件，程序不需要输出任何内容。对于那些存在语义错误的输入文件，程序应当输出相应的错误信息，这些信息包括错误类型、出错的行号以及说明文字，其格式为：

```
Error type [错误类型] at Line [行号]: [说明文字].
```

说明文字的内容没有具体要求，但是错误类型和出错的行号一定要正确，因为这是判断输出的错误提示信息是否正确的唯一标准。请严格遵守实践内容要求中给定的错误分类，否则将影响实践内容评分。

输入文件中可能包含一个或者多个错误（但每行最多只有一个错误），程序需要将它们全部检查出来。当然，有些时候输入文件中的一个错误会产生连锁反应，导致别的地方出现多个错误（例如，一个未定义的变量在使用时由于无法确定其类型，会使所有包含该变量的表达式产生类型错误），我们只会去考察程序是否报告了本质错误（如果难以确定哪个错误是本质错误，建议报告所有发现的错误）。但是，如果源程序里有错而程序没有报错或报告的错误类型不对，又或者源程序里没有错但程序却报错，都会影响实践内容评分。

3.3.4 验证环境

程序将在如下环境中被编译并运行（同实践内容一）：

- （1）GNU Linux Release: Ubuntu 20.04, kernel version 5.13.0-44-generic;
- （2）GCC version 7.5.0;
- （3）GNU Flex version 2.6.4;
- （4）GNU Bison version 3.5.1。

一般而言，只要避免使用过于冷门的特性，使用其它版本的 Linux 或者 GCC 等，也基本上不会出现兼容性方面的问题。注意，实践内容二的检查过程中不会去安装或尝试引用各类方便编程的函数库（如 glib 等），因此请不要在程序中使用它们。

3.3.5 提交要求

实践内容二要求提交如下内容（同实践内容一）：

(1) Flex、Bison 以及 C 语言的可被正确编译运行的源程序。

(2) 一份 PDF 格式的实践内容报告，内容包括：

(a) 程序实现了哪些功能？简要说明如何实现这些功能。清晰的说明有助于助教对程序所实现的功能进行合理的测试。

(b) 程序应该如何被编译？可以使用脚本、makefile 或逐条输入命令进行编译，请详细说明应该如何编译程序。无法顺利编译将导致助教无法对程序所实现的功能进行任何测试，从而丢失相应的分数。

(c) 实践内容报告的长度不得超过三页！所以实践内容报告中需要重点描述的是程序中的亮点，是提交者认为最个性化、最具独创性的内容，而相对简单的、任何人都可以做的内容则可不提或简单地提一下，尤其要避免大段地向报告里贴代码。实践内容报告中所出现的最小字号不得小于 5 号字（或英文 11 号字）。

3.3.6 样例（必做部分）

实践内容二的样例包括必做内容样例与选做要求样例两部分，分别对应于实践内容要求中的必做内容和选做要求。请仔细阅读样例，以加深对实践内容要求以及输出格式要求的理解。本节列举必做内容样例。

样例 1:

输入:

```
1 int main()  
2 {  
3     int i = 0;  
4     j = i + 1;  
5 }
```

输出:

样例输入中变量“j”未定义，因此程序可以输出如下的错误提示信息：

```
Error type 1 at Line 4: Undefined variable "j".
```

样例 2:

输入:

```
1 int main()
2 {
3     int i = 0;
4     inc(i);
5 }
```

输出:

样例输入中函数“inc”未定义，因此程序可以输出如下的错误提示信息:

Error type 2 at Line 4: Undefined function "inc".

样例 3:

输入:

```
1 int main()
2 {
3     int i, j;
4     int i;
5 }
```

输出:

样例输入中变量“i”被重复定义，因此程序可以输出如下的错误提示信息:

Error type 3 at Line 4: Redefined variable "i".

样例 4:

输入:

```
1 int func(int i)
2 {
3     return i;
4 }
5
6 int func()
7 {
8     return 0;
9 }
10
11 int main()
12 {
13 }
```

输出:

样例输入中函数“func”被重复定义，因此程序可以输出如下的错误提示信息:

Error type 4 at Line 6: Redefined function "func".

样例 5:

输入:

```
1 int main()
2 {
3     int i;
4     i = 3.7;
5 }
```

输出:

样例输入中错将一个浮点常数赋值给一个整型变量，因此程序可以输出如下的错误提示信息:

Error type 5 at Line 4: Type mismatched for assignment.

样例 6:

输入:

```
1 int main()
2 {
3     int i;
4     10 = i;
5 }
```

输出:

样例输入中整数“10”出现在了赋值号的左边，因此程序可以输出如下的错误提示信息:

Error type 6 at Line 4: The left-hand side of an assignment must be a variable.

样例 7:

输入:

```
1 int main()
2 {
3     float j;
4     10 + j;
5 }
```

输出:

样例输入中表达式“10+j”的两个操作数的类型不匹配，因此程序可以输出如下的错误提示信息:

Error type 7 at Line 4: Type mismatched for operands.

样例 8:

输入:

```
1 int main()
2 {
3     float j = 1.7;
4     return j;
```

```
5 }
```

输出:

样例输入中“main”函数返回值的类型不正确, 因此程序可以输出如下的错误提示信息:

```
Error type 8 at Line 4: Type mismatched for return.
```

样例 9:

输入:

```
1 int func(int i)
2 {
3     return i;
4 }
5
6 int main()
7 {
8     func(1, 2);
9 }
```

输出:

样例输入中调用函数“func”时实参数目不正确, 因此程序可以输出如下的错误提示信息:

```
Error type 9 at Line 8: Function "func(int)" is not applicable for arguments
"(int, int)".
```

样例 10:

输入:

```
1 int main()
2 {
3     int i;
4     i[0];
5 }
```

输出:

样例输入中变量“i”不是数组型变量, 因此程序可以输出如下的错误提示信息:

```
Error type 10 at Line 4: "i" is not an array.
```

样例 11:

输入:

```
1 int main()
2 {
3     int i;
4     i(10);
5 }
```

输出:

样例输入中变量“i”不是函数，因此程序可以输出如下的错误提示信息：

```
Error type 11 at Line 4: "i" is not a function.
```

样例 12:

输入：

```
1 int main()
2 {
3     int i[10];
4     i[1.5] = 10;
5 }
```

输出：

样例输入中数组访问符中出现了非整型常数“1.5”，因此程序可以输出如下的错误提示信息：

```
Error type 12 at Line 4: "1.5" is not an integer.
```

样例 13:

输入：

```
1 struct Position
2 {
3     float x, y;
4 };
5
6 int main()
7 {
8     int i;
9     i.x;
10 }
```

输出：

样例输入中变量“i”不是结构体类型变量，因此程序可以输出如下的错误提示信息：

```
Error type 13 at Line 9: Illegal use of ".".
```

样例 14:

输入：

```
1 struct Position
2 {
3     float x, y;
4 };
5
6 int main()
7 {
8     struct Position p;
9     if (p.n == 3.7)
10         return 0;
11 }
```

输出:

样例输入中结构体变量“p”访问了未定义的域“n”，因此程序可以输出如下的错误信息:

```
Error type 14 at Line 9: Non-existent field "n".
```

样例 15:

输入:

```
1 struct Position
2 {
3     float x, y;
4     int x;
5 };
6
7 int main()
8 {
9 }
```

输出:

样例输入中结构体的域“x”被重复定义，因此程序可以输出如下的错误信息:

```
Error type 15 at Line 4: Redefined field "x".
```

样例 16:

输入:

```
1 struct Position
2 {
3     float x;
4 };
5
6 struct Position
7 {
8     int y;
9 };
10
11 int main()
12 {
13 }
```

输出:

样例输入中两个结构体的名字重复，因此程序可以输出如下的错误信息:

```
Error type 16 at Line 6: Duplicated name "Position".
```

样例 17:

输入:

```
1 int main()
2 {
```

```
3  struct Position pos;
4  }
```

输出:

样例输入中结构体“Position”未经定义，因此程序可以输出如下的错误信息:

Error type 17 at Line 3: Undefined structure "Position".

3.3.7 样例（选做部分）

这节列举选做要求样例。

样例 1:

输入:

```
1  int func(int a);
2
3  int func(int a)
4  {
5      return 1;
6  }
7
8  int main()
9  {
10 }
```

输出:

如果程序需要完成要求 3.1，这个样例输入不存在任何词法、语法或语义错误，因此不需要输出错误信息。

如果程序不需要完成要求 3.1，这个样例输入存在语法错误，因此程序可以输出如下的错误提示信息:

Error type B at Line 1: Incomplete definition of function "func".

样例 2:

输入:

```
1  struct Position
2  {
3      float x,y;
4  };
5
6  int func(int a);
7
8  int func(struct Position p);
9
10 int main()
11 {
```

```
12 }
```

输出:

如果程序需要完成要求 3.1, 这个样例输入存在两处语义错误: 一是函数 “func” 的两次声明不一致; 二是函数 “func” 未定义, 因此程序可以输出如下的错误提示信息:

```
Error type 19 at Line 8: Inconsistent declaration of function "func".
```

```
Error type 18 at Line 6: Undefined function "func".
```

注意, 我们对错误提示信息的顺序不做要求。

如果程序不需要完成要求 3.1, 这个样例输入存在两处语法错误, 因此程序可以输出如下的错误提示信息:

```
Error type B at Line 6: Incomplete definition of function "func".
```

```
Error type B at Line 8: Incomplete definition of function "func".
```

样例 3:

输入:

```
1 int func()
2 {
3     int i = 10;
4     return i;
5 }
6
7 int main()
8 {
9     int i;
10    i = func();
11 }
```

输出:

如果程序需要完成要求 3.2, 这个样例输入不存在任何词法、语法或语义错误, 因此不需要输出错误信息。

如果程序不需要完成要求 3.2, 样例输入中的变量 “i” 被重复定义, 因此程序可以输出如下的错误信息:

```
Error type 3 at Line 9: Redefined variable "i".
```

样例 4:

输入:

```
1 int func()
2 {
3     int i = 10;
4     return i;
5 }
6
7 int main()
8 {
9     int i;
10    int i, j;
11    i = func();
12 }
```

输出:

如果程序需要完成要求 3.2, 样例输入中的变量“i”被重复定义, 因此程序可以输出如下的错误提示信息:

```
Error type 3 at Line 10: Redefined variable "i".
```

如果程序不需要完成要求 3.2, 样例输入中的变量“i”被重复定义了两次, 因此程序可以输出如下的错误提示信息:

```
Error type 3 at Line 9: Redefined variable "i".
Error type 3 at Line 10: Redefined variable "i".
```

样例 5:

输入:

```
1 struct Temp1
2 {
3     int i;
4     float j;
5 };
6
7 struct Temp2
8 {
9     int x;
10    float y;
11 };
12
13 int main()
14 {
15     struct Temp1 t1;
16     struct Temp2 t2;
17     t1 = t2;
18 }
```

输出:

如果程序需要完成要求 3.3, 这个样例输入不存在任何词法、语法或语义错误, 因此不需要输出错误信息。